

AI-Driven Log Analysis and Diagnostics Platform

Scalable System Design for Distributed Infrastructure
Processing ~1 TB of Log Data Daily

System Design Presentation

Yash Lohia

1. Problem Statement

Design a large-scale AI-driven log analysis and diagnostics system for a distributed infrastructure environment consisting of thousands of virtual machines, each generating approximately 1 GB of logs per day - resulting in nearly 1 TB of log data daily.

Log Sources

- Console logs - stdout/stderr output from running applications
- System logs - OS-level events (syslog, kernel logs, auth logs)
- Application logs - Structured/unstructured output from application code

Additional Data Sources

- Application source code repositories
- Historical incident and resolution data
- Build and deployment metadata (optional)

Core Challenge

The system must automatically ingest, process, and analyze massive volumes of heterogeneous log data in real-time, detect anomalies and errors, correlate events across distributed systems, determine root causes, and recommend remediation - all with minimal human intervention.

2. High-Level Architecture

The platform follows a layered, microservices-based architecture with six core layers. Each layer is independently scalable and communicates through well-defined interfaces.



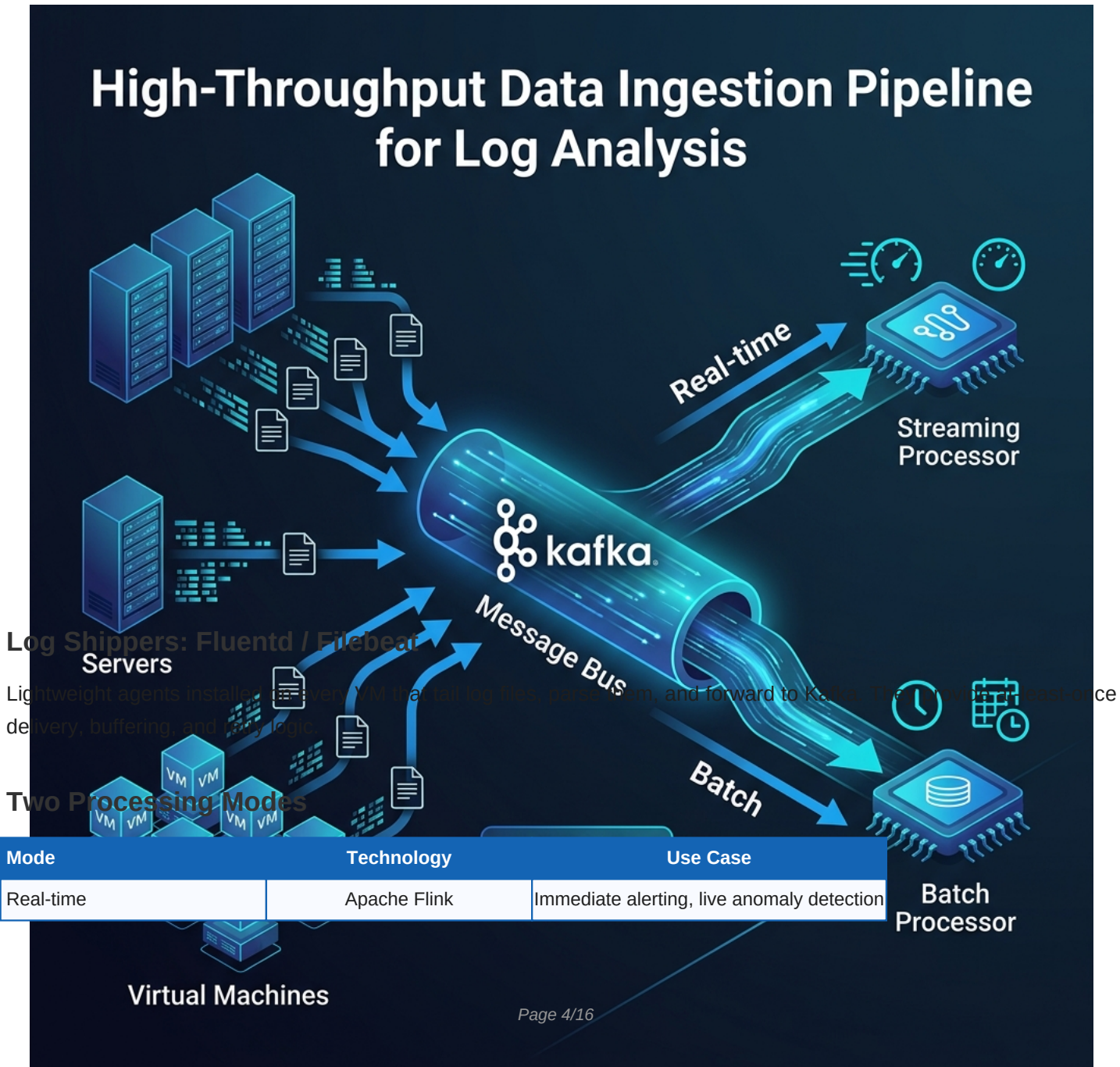
3. Data Ingestion Layer

Design Choice: Apache Kafka as Central Message Bus

At ~1 TB/day from thousands of VMs, no single machine can handle the load. Apache Kafka provides a distributed, fault-tolerant message queue that acts as the backbone of the entire pipeline.

Why Kafka?

- **High Throughput:** Handles millions of messages per second across partitioned topics
- **Durability:** Messages are persisted to disk with configurable replication (factor of 3)
- **Replay Capability:** Consumers can re-read data from any offset - critical for reprocessing
- **Multiple Consumers:** Stream processor, batch processor, and archiver can all read independently
- **Backpressure:** Naturally handles traffic spikes without data loss



Batch	Apache Spark	Deep pattern mining, model training, reports
-------	--------------	--

4. Log Processing & Normalization

Logs arrive in heterogeneous formats - syslog, JSON, plain text, CSV. The processing layer parses and normalizes all formats into a unified schema.

Example: Heterogeneous Input

```
# Syslog format
May 20 10:30:01 vm-1234 sshd[12345]: Failed password for root

# JSON application log
{"timestamp": "2026-05-20T10:30:00Z", "level": "ERROR", "msg": "DB timeout"}

# Plain text
[ERROR] 2026-05-20 10:30:00 - NullPointerException at UserService.java:142
```

Unified Output Schema

```
{
  "timestamp": "2026-05-20T10:30:00Z",
  "source_host": "vm-1234",
  "service": "auth-service",
  "log_level": "ERROR",
  "message": "DB connection timeout",
  "metadata": {
    "region": "us-east-1",
    "deployment_version": "v2.3.1",
    "cluster": "prod-cluster-a"
  }
}
```

Enrichment

After parsing, logs are enriched with contextual metadata from external sources:

- Deployment metadata: Which version is running on this VM?
- Topology data: What service dependency graph does this VM belong to?
- Infrastructure metadata: Region, cluster, availability zone

5. Detection and Analysis

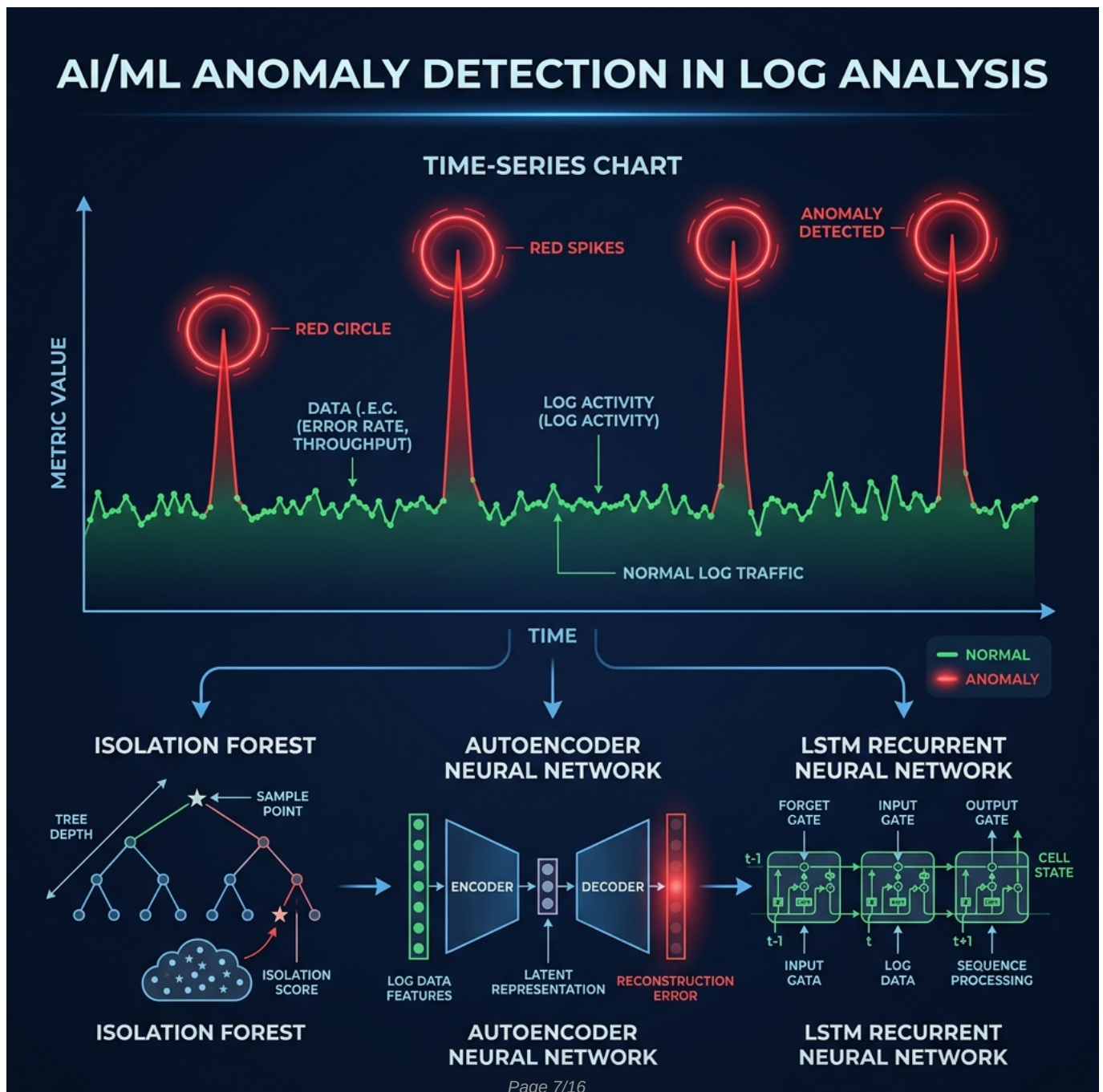
A. Rule-Based Detection

Pattern matching for known issue categories:

- Log level ERROR or FATAL -> immediate alert
- HTTP 5xx status codes -> server error detected
- Keywords: "OutOfMemoryError", "timeout", "connection refused"
- Threshold breaches: CPU > 90%, disk > 95%, response time > 5s

B. ML-Based Anomaly Detection

For issues not caught by static rules:



Isolation Forest

Unsupervised algorithm that isolates anomalies by randomly partitioning data. Anomalous points (e.g., sudden error spikes) are isolated in fewer partitions. Used for detecting unusual log volume patterns.

Autoencoders (Neural Networks)

Trained to reconstruct "normal" log patterns. When a new log event cannot be reconstructed accurately (high reconstruction error), it is flagged as anomalous. Effective for detecting novel, previously unseen failure modes.

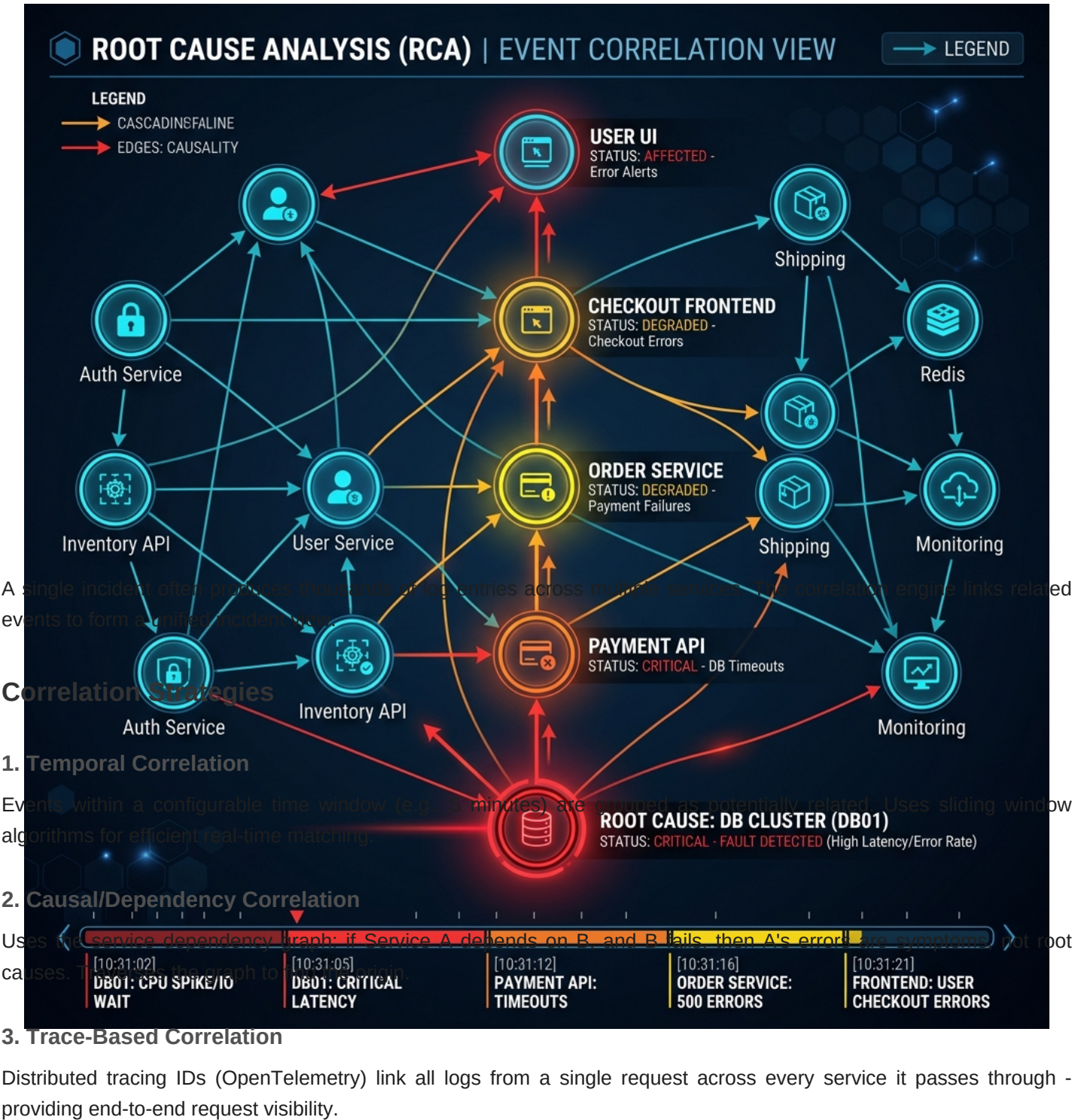
Time-Series Analysis (LSTM)

Monitors metrics derived from logs (error rates, latency trends) over time. Detects gradual degradation patterns like slowly increasing response times that rule-based systems miss.

Log Clustering (NLP)

Uses TF-IDF and embedding-based clustering to group similar log messages. Discovers new error categories automatically without manual rule creation.

6. Correlation Engine



Example: Correlated Incident

10:30:00	- Database:	"Disk space critical (98% full)"
10:30:05	- Database:	"Write operation failed"
10:30:06	- Auth Service:	"DB connection timeout"
10:30:07	- API Gateway:	"502 Bad Gateway from auth-service"

10:30:07 - Frontend: "User login failed"

Root Cause: Database disk space -> all other errors are symptoms

7. Root Cause Analysis (RCA)

Approach 1: Dependency Graph Traversal

Build a directed graph of service dependencies (A -> B -> C -> DB). When multiple services fail, trace back to the deepest failing node in the dependency chain. That node is the most likely root cause.

Approach 2: Bayesian / Statistical Inference

Calculate $P(\text{root_cause} \mid \text{observed_symptoms})$ using historical incident data. Example: "When auth-service and api-gateway both timeout, the database was the root cause 73% of the time in past incidents."

Approach 3: Historical Pattern Matching

Compare the current incident fingerprint (set of correlated events) against a database of past incidents using similarity metrics. If the current pattern is 92% similar to a past incident, surface that incident's root cause and resolution.

Code Mapping

Once the root cause is identified, map it to source code:

- Stack trace parsing: extract file names, line numbers, function names
- Repository indexing: link "UserService.java:142" to the actual code
- Change correlation: check recent git commits to the failing module

8. Remediation & Resolution Assistance

RAG-Based Recommendation Engine

Uses Retrieval-Augmented Generation (RAG) to suggest fixes:

1. Store all past incident reports + resolutions in a vector database (FAISS)
2. When new incident occurs, embed the incident description
3. Retrieve top-K most similar past incidents
4. Feed context to an LLM to generate remediation steps
5. Present ranked recommendations to the operator

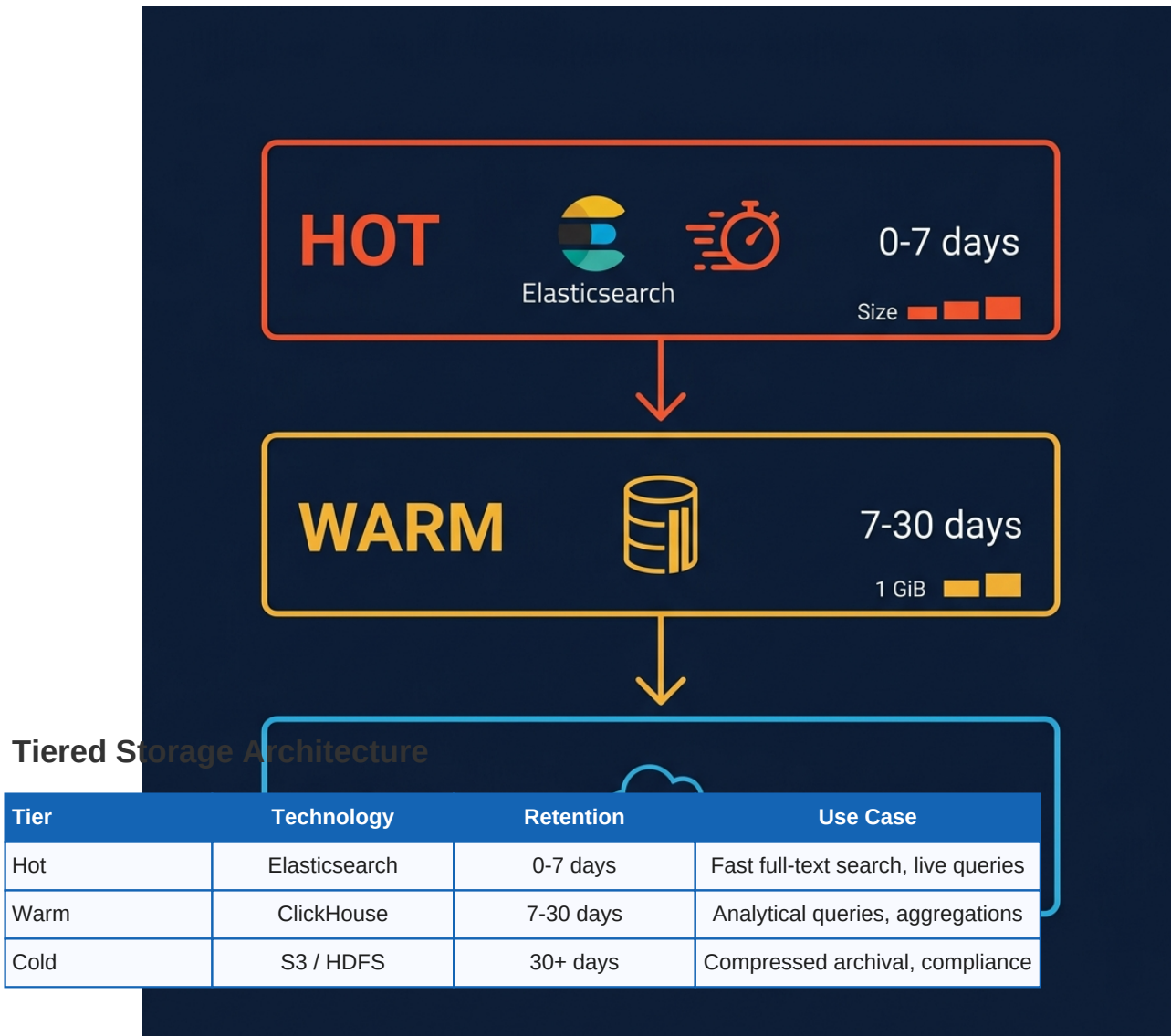
Types of Recommendations

- **Configuration Fixes:** "Increase max_connections from 100 to 500 in db.conf"
- **Operational Actions:** "Run docker system prune to free disk space"
- **Code Changes:** "Add retry logic with exponential backoff in UserService.java:142"
- **Infrastructure:** "Scale up database instance or add read replicas"

Automated Validation

Where applicable, trigger CI/CD pipelines to automatically test suggested fixes before deploying to production. Integration with Jenkins/GitHub Actions.

9. Storage Strategy



Why This Approach?

- Hot tier (Elasticsearch): Optimized for real-time search and dashboards
- Warm tier (ClickHouse): Columnar storage, 10x compression, fast analytical queries
- Cold tier (S3): Cheapest storage, rarely accessed, used for compliance and ML training

Data Lifecycle

```
Log arrives -> Kafka -> Flink processes in real-time
-> Indexed in Elasticsearch (hot, 7 days)
-> After 7 days, migrated to ClickHouse (warm, 30 days)
-> After 30 days, compressed and archived to S3 (cold, indefinite)
```

10. Technology Stack

Layer	Technology	Justification
Log Collection	Fluentd / Filebeat	Lightweight, reliable log shippers
Message Queue	Apache Kafka	High throughput, durability, replay
Stream Processing	Apache Flink	Low latency, exactly-once semantics
Batch Processing	Apache Spark	Large-scale analytics and ML training
Hot Storage	Elasticsearch	Full-text search on recent logs
Warm Storage	ClickHouse	Columnar DB for analytical queries
Cold Storage	S3 / HDFS	Cost-effective archival
Anomaly Detection	scikit-learn/PyTorch	Isolation Forest, Autoencoders
Correlation	Neo4j	Dependency graph storage/traversal
LLM / RAG	LLM + FAISS	Remediation suggestion generation
Dashboard	Grafana / React	Real-time visualization
Orchestration	Kubernetes	Container orchestration at scale
CI/CD	Jenkins / GH Actions	Automated validation workflows

11. Scalability Considerations

Horizontal Scaling Strategy

- **Kafka:** Add brokers + partitions as log volume grows
- **Flink:** Add task managers for parallel stream processing
- **Elasticsearch:** Add data nodes, use index lifecycle management
- **ML Models:** Multiple replicas behind a load balancer

Estimated Resources (~1 TB/day)

Component	Sizing
Kafka Cluster	5-10 brokers, 3x replication
Flink Cluster	10-20 task managers
Elasticsearch	5-10 data nodes, ~30 TB (7d)
Total Platform	~50-100 VMs

Reliability & Fault Tolerance

Concern	Solution
Log loss	Kafka durability + at-least-once delivery
Processing failure	Flink checkpointing + auto-restart
Storage node failure	ES/Kafka replication (factor 3)
ML model failure	Multiple replicas + load balancer
Network partition	Kafka handles natively; offset-based resume

12. End-to-End Pipeline Summary

Step 1: Ingest logs from distributed systems (Fluentd -> Kafka)
Step 2: Parse and normalize into unified schema (Flink)
Step 3: Detect errors, warnings, anomalies (Rules + ML)
Step 4: Correlate events across systems (Temporal + Causal + Trace)
Step 5: Root-cause analysis (Dependency graph + Bayesian + Historical)
Step 6: Identify impacted systems and components
Step 7: Map issues to source code areas (Stack traces + Git)
Step 8: Generate remediation recommendations (RAG + LLM)
Step 9: Trigger automated build/test validation (CI/CD)
Step 10: Continuously learn from resolved incidents (Feedback loop)

13. Conclusion

This platform design addresses all functional requirements of the AI-driven log analysis problem through a layered, horizontally scalable architecture. Key design decisions include:

- Apache Kafka as the central nervous system for reliable, high-throughput ingestion
- Dual processing modes (Flink streaming + Spark batch) for real-time and deep analysis
- Hybrid detection using rule engines for known patterns and ML for novel anomalies
- Multi-strategy correlation (temporal, causal, trace-based) for accurate incident grouping
- Probabilistic RCA using dependency graphs, Bayesian inference, and historical matching
- RAG-based remediation engine leveraging past incident resolutions
- Tiered storage (ES/ClickHouse/S3) balancing query speed with cost efficiency
- Continuous learning loop that improves diagnostics with each resolved incident

The architecture demonstrates competency across distributed systems design, data pipeline engineering, AI/ML integration, observability, scalability, reliability, and practical automation workflows - covering all evaluation areas specified in the problem statement.